

Техническое интервью Mobile React Native Developer: Вопросы и Ответы

1. Что такое React Native и чем он отличается от React?

React Native — это фреймворк для создания мобильных приложений с использованием React и JavaScript. В то время как React предназначен для веб-браузеров и использует HTML/CSS, React Native предназначен для мобильных платформ (iOS и Android) и использует нативные компоненты. React Native компилируется в нативный код, обеспечивая производительность, близкую к нативной. Основное отличие заключается в том, что React Native использует нативные мобильные компоненты (View, Text, Image) вместо веб-элементов (div, span, img), и использует API StyleSheet вместо CSS. Это позволяет делиться кодом между платформами, сохраняя при этом нативный внешний вид и поведение.

2. Объясните архитектуру React Native и как она работает.

React Native использует архитектуру моста, где код JavaScript выполняется в отдельном потоке и взаимодействует с нативными модулями через асинхронный мост. Поток JavaScript обрабатывает бизнес-логику и компоненты React, в то время как нативные потоки обрабатывают рендеринг UI и платформо-специфичные API. Когда JavaScript должен обновить UI, он отправляет сериализованные сообщения через мост в нативные модули. Эта архитектура позволяет React Native использовать нативную производительность, используя JavaScript для разработки. Новые архитектуры (Fabric и TurboModules) уменьшают накладные расходы моста для лучшей производительности.

3. В чем разница между React Native и нативной мобильной разработкой?

Нативная разработка использует платформо-специфичные языки (Swift/Objective-C для iOS, Kotlin/Java для Android) и обеспечивает лучшую производительность и полный доступ к функциям платформы. React Native использует JavaScript и позволяет делиться кодом между платформами, сокращая время разработки и обслуживания. Нативные приложения обычно имеют лучшую производительность и доступ к последним функциям платформы сразу, в то время как React Native предлагает более быструю разработку, более простое обслуживание и единую кодовую базу. Выбор зависит от требований к производительности, экспертизы команды и временных рамок разработки.

4. Объясните жизненный цикл компонентов React Native и хуки.

Компоненты React Native следуют тому же жизненному циклу, что и веб-компоненты React. Классовые компоненты используют методы жизненного цикла, такие как `componentDidMount`, `componentDidUpdate` и `componentWillUnmount`. Функциональные компоненты используют хуки: `useState` для состояния, `useEffect` для побочных эффектов (эквивалент `componentDidMount/Update`), `useCallback` и `useMemo` для оптимизации, и пользовательские хуки для переиспользуемой логики. `useEffect` может обрабатывать монтирование, обновление и очистку, возвращая функцию очистки. Понимание жизненного цикла важно для правильного управления ресурсами, подписками и вызовами API.

5. Как вы обрабатываете навигацию в React Native?

Навигация в React Native обычно обрабатывается React Navigation, самой популярной библиотекой. Она предоставляет стековую навигацию (стек экранов), табовую навигацию (нижние/верхние вкладки), навигацию с боковым меню (drawer) и вложенные навигаторы. React Navigation использует декларативный API, где вы определяете структуру навигации, и она обрабатывает переходы, глубокие ссылки и управление состоянием. Альтернативные библиотеки включают React Native Navigation (более нативная, лучшая производительность) и React Router Native. Состояние навигации управляется централизованно, позволяя программную навигацию и передачу параметров между экранами.

6. В чем разница между state и props в React Native?

Props — это данные только для чтения, передаваемые от родительского компонента к дочернему, аналогично параметрам функции. Они неизменяемы и не могут быть изменены дочерним компонентом. State — это изменяемые данные, управляемые внутри компонента с помощью хука `useState`. Когда состояние изменяется, React Native перерисовывает компонент. Props передаются вниз по дереву компонентов, в то время как state является локальным, если не поднят к общему предку. Для глобального состояния вы бы использовали Context API, Redux или другие решения для управления состоянием. Понимание этого различия является фундаментальным для разработки на React Native.

7. Как вы обрабатываете вызовы API и получение данных в React Native?

Вызовы API в React Native используют `fetch()` или библиотеки, такие как `axios`, аналогично веб-React. Получение данных обычно происходит в хуках `useEffect` с правильными массивами зависимостей. Вы управляете состояниями загрузки, ошибками и данными, используя `useState`. Для более сложных сценариев библиотеки, такие как `React Query` или `SWR`, предоставляют кэширование, фоновое обновление и лучшее управление состоянием. Важно обрабатывать сетевые ошибки, реализовывать логику повторов и показывать соответствующие индикаторы загрузки. Для поддержки офлайн-режима вы можете использовать библиотеки, такие как `React Query` с персистентностью, или реализовать пользовательские стратегии кэширования.

8. Объясните API StyleSheet в React Native.

React Native использует `StyleSheet.create()` вместо CSS файлов. Стили — это объекты JavaScript со свойствами в camelCase (`backgroundColor` вместо `background-color`). `StyleSheet` обеспечивает оптимизацию производительности, создавая объекты стилей один раз и переиспользуя их, и проверяет стили во время создания. Вы можете комбинировать стили, используя массивы, и стили каскадируются, но не поддерживают все функции CSS. `Flexbox` — это основная система макета, и вы можете использовать абсолютное позиционирование. Платформенно-специфичные стили могут быть применены с использованием проверок `Platform.OS` или `Platform.select()`.

9. Какие техники оптимизации производительности React Native?

Оптимизация производительности включает: использование `React.memo` для предотвращения ненужных перерисовок, реализацию `FlatList` или `SectionList` для длинных списков (вместо `ScrollView` с `map`), использование `useMemo` и `useCallback` для дорогих вычислений, оптимизацию изображений (изменение размера, использование подходящих форматов), уменьшение коммуникации через мост, использование нативного драйвера для анимаций, реализацию разделения кода и избегание встроенных функций и создания объектов в `render`. Движок JavaScript Hermes (включен по умолчанию в новых версиях) также улучшает время запуска и использование памяти.

10. Как вы обрабатываете изображения в React Native?

React Native предоставляет компонент `Image` для отображения изображений. Вы можете загружать изображения из локальных ресурсов (`require('./image.png')`), сетевых URL или данных base64. Для сетевых изображений вы должны обрабатывать состояния загрузки и ошибки. Компонент `Image` поддерживает `resizeMode` (`cover`, `contain`, `stretch`, `repeat`, `center`) для различных потребностей отображения. Для лучшей производительности вы можете использовать библиотеки, такие как `react-native-fast-image` для кэширования и оптимизации. Важно оптимизировать размеры изображений и использовать подходящие форматы (WebP для Android, HEIC для iOS, когда возможно).

11. Объясните управление состоянием в приложениях React Native.

Варианты управления состоянием включают: локальное состояние (`useState` для данных, специфичных для компонента), `Context API` (для обмена данными между компонентами без `prop drilling`), `Redux/Redux Toolkit` (для сложного глобального состояния), `Zustand` или `Jotai` (более легкие альтернативы `Redux`) и `React Query` (для состояния сервера). Выбор зависит от сложности приложения — простые приложения могут нуждаться только в `useState` и `Context`, в то время как сложные приложения выигрывают от `Redux` или подобных решений. `React Query` отлично подходит для состояния сервера (данные API), так как автоматически обрабатывает кэширование, синхронизацию и фоновые обновления.

12. Что такое мосты React Native и нативные модули?

Мосты обеспечивают связь между JavaScript и нативным кодом. Когда React Native не имеет встроенного API для чего-то, вы создаете нативные модули — код, написанный на нативных языках (Swift/Objective-C для iOS, Kotlin/Java для Android), который может быть вызван из JavaScript. Нативные модули полезны для доступа к платформо-специфичным функциям, таким как камера, Bluetooth, биометрия или сторонние SDK. Вы определяете интерфейс в нативном коде, создаете мост и экспонируете его в JavaScript. Библиотеки, такие как `react-native-create-bridge`, упрощают этот процесс.

13. Как вы обрабатываете офлайн-функциональность в React Native?

Офлайн-функциональность может быть реализована с использованием: AsyncStorage или MMKV для локального хранения данных, NetInfo для определения статуса сети, React Query с персистентностью для кэширования ответов API, Redux Persist для персистентности состояния и реализации логики синхронизации для загрузки изменений, когда онлайн. Вы должны кэшировать критические данные, ставить действия в очередь, когда офлайн, и синхронизировать, когда возвращается подключение. Библиотеки, такие как WatermelonDB или Realm, предоставляют локальные базы данных для сложных офлайн-сценариев. Правильная обработка офлайн-режима значительно улучшает пользовательский опыт.

14. Объясните разницу между FlatList и ScrollView.

ScrollView рендерит все дочерние элементы сразу, что подходит для небольших списков, но вызывает проблемы с производительностью при большом количестве элементов. FlatList виртуализирован — он рендерит только видимые элементы и перерабатывает представления при прокрутке, что делает его эффективным для длинных списков. FlatList предоставляет встроенные функции, такие как pull-to-refresh, индикаторы загрузки, пустые состояния и заголовки секций. Используйте ScrollView для небольшого статического контента и FlatList для динамических списков данных. SectionList расширяет FlatList для сгруппированных данных с заголовками секций.

15. Как вы обрабатываете формы и пользовательский ввод в React Native?

Формы в React Native используют компоненты TextInput, управляемые состоянием. Вы управляете значениями ввода с помощью useState и обрабатываете изменения с помощью onChangeText. Для сложных форм библиотеки, такие как Formik или React Hook Form, предоставляют валидацию, обработку ошибок и управление состоянием формы. TextInput поддерживает различные пропсы, такие как placeholder, keyboardType, secureTextEntry, multiline и autoCapitalize. Вы должны обрабатывать различные типы клавиатур, реализовывать правильную валидацию, показывать сообщения об ошибках и управлять фокусом между вводами. Для лучшего UX рассмотрите использование библиотек, таких как react-native-paper или native-base, которые предоставляют улучшенные компоненты форм.

16. Что такое движок JavaScript Hermes и почему он важен?

Hermes — это движок JavaScript, оптимизированный для React Native, разработанный Facebook. Он улучшает производительность приложения за счет: уменьшения использования памяти, сокращения времени запуска приложения и обеспечения лучших метрик time-to-interactive. Hermes компилирует JavaScript в байт-код заранее, что быстрее парсится и выполняется. Он включен по умолчанию в React Native 0.70+ и обеспечивает значительные улучшения производительности, особенно на Android. Hermes делает приложения React Native более нативными и отзывчивыми.

17. Как вы обрабатываете push-уведомления в React Native?

Push-уведомления требуют: настройки Firebase Cloud Messaging (FCM) для Android и Apple Push Notification service (APNs) для iOS, настройки нативных разрешений и использования библиотек, таких как react-native-push-notification или @react-native-firebase/messaging. Вам нужно запрашивать разрешения на уведомления, обрабатывать регистрацию токенов, настраивать обработчики уведомлений для состояний foreground/background/quit и реализовывать глубокие ссылки, если уведомления должны открывать определенные экраны. Тестирование требует физических устройств или правильной конфигурации симулятора, так как push-уведомления не работают в стандартных симуляторах.

18. Объясните code pushing и обновления over-the-air (OTA) в React Native.

Code pushing позволяет обновлять код JavaScript без прохождения проверки в магазине приложений, используя сервисы, такие как CodePush (Microsoft) или EAS Update (Expo). Это позволяет быстро исправлять ошибки и развертывать функции. Однако вы не можете обновлять нативный код — только JavaScript бандлы. OTA-обновления полезны для: быстрого исправления критических ошибок, A/B тестирования, постепенного развертывания функций и обновления бизнес-логики. Изменения нативного кода все еще требуют обновлений через магазин приложений. CodePush интегрируется с CI/CD пайплайнами и поддерживает поэтапные развертывания и откаты.

19. Как вы обрабатываете глубокие ссылки в React Native?

Глубокие ссылки позволяют открывать определенные экраны приложения через URL. Реализация включает: настройку URL-схем (myapp://) или универсальных ссылок (https://myapp.com),

обработку входящих ссылок в приложении, парсинг параметров URL и навигацию к соответствующим экранам. Библиотеки, такие как react-native-deep-linking или конфигурация linking в React Navigation, упрощают это. Вам нужно обрабатывать случаи, когда приложение закрыто, в фоне или уже открыто. Глубокие ссылки необходимы для: подтверждения email, сброса пароля, обмена контентом и маркетинговых кампаний, направляющих пользователей к определенным разделам приложения.

20. Какие паттерны платформно-специфичного кода в React Native?

Платформно-специфичный код может быть реализован с использованием: Platform.OS для проверки платформы ('ios' или 'android'), Platform.select() для платформо-специфичных значений, отдельных файлов с расширениями .ios.js и .android.js и условного рендеринга. Это позволяет вам предоставлять различные реализации, стили или компоненты для каждой платформы, делясь при этом большей частью кодовой базы. Платформно-специфичный код необходим, когда: используются функции, эксклюзивные для платформы, реализуется различный UI/UX в соответствии с руководящими принципами платформы или обрабатываются платформо-специфичные API.

21. Как вы обрабатываете анимации в React Native?

React Native предоставляет API Animated для анимаций, управляемых JavaScript, и react-native-reanimated для лучшей производительности с worklets, выполняющимися в потоке UI. API Animated проще, но выполняется в потоке JS, в то время как Reanimated 2 выполняет анимации в потоке UI для производительности 60fps. Для анимаций макета вы можете использовать LayoutAnimation. Общие случаи использования анимаций включают: переходы между экранами, анимации на основе жестов, индикаторы загрузки и микро-взаимодействия. Reanimated рекомендуется для сложных анимаций и когда производительность критична.

22. Объясните обработку ошибок и отчетность о сбоях в React Native.

Обработка ошибок включает: использование блоков try-catch для синхронного кода, обработку отклонений промисов, реализацию error boundaries (хотя ограниченно в React Native) и использование глобальных обработчиков ошибок. Для отчетности о сбоях сервисы, такие как Sentry, Bugsnag или Crashlytics, захватывают сбои, предоставляют трассировки стека и помогают

отлаживать проблемы в продакшене. Вы должны логировать ошибки соответствующим образом, предоставлять понятные пользователю сообщения об ошибках и реализовывать логику повторов для сетевых сбоев. Правильная обработка ошибок и отчетность важны для поддержания стабильности приложения и быстрого исправления проблем.

23. Как вы тестируете приложения React Native?

Стратегии тестирования включают: модульные тесты с Jest для бизнес-логики, тесты компонентов с React Native Testing Library, интеграционные тесты для потоков функций и E2E тесты с Detox или Appium. Jest настроен по умолчанию и может тестировать код JavaScript. React Native Testing Library предоставляет утилиты для тестирования компонентов. Detox позволяет E2E тестирование на симуляторах/эмуляторах. Тестирование нативных модулей требует дополнительной настройки. Вы должны тестировать критические пользовательские потоки, граничные случаи и обеспечивать запуск тестов в CI/CD пайплайнах. Хорошее покрытие тестами помогает выявлять регрессии и позволяет уверенно рефакторить.

24. В чем разница между Expo и React Native CLI?

Expo — это фреймворк и платформа, построенная вокруг React Native, которая упрощает разработку с: управляемым workflow (изначально без доступа к нативному коду), встроенными API и сервисами, обновлениями over-the-air, более простыми сборками и развертываниями и обширной документацией. React Native CLI предоставляет полный контроль над нативным кодом с самого начала, но требует больше настройки и знаний нативной разработки. Expo отлично подходит для быстрого прототипирования и приложений, которым не нужны пользовательские нативные модули, в то время как CLI лучше, когда вам нужен полный нативный контроль или специфичные нативные библиотеки, не поддерживаемые Expo.

25. Как вы обрабатываете персистентность состояния приложения в React Native?

Персистентность состояния может быть реализована с использованием: AsyncStorage для простого хранилища ключ-значение, MMKV для лучшей производительности, Redux Persist для состояния Redux, React Query с персистентностью для состояния сервера или баз данных, таких как WatermelonDB или Realm, для сложных данных. Вы должны сохранять: токены аутентификации пользователя, пользовательские настройки, кэшированные данные и офлайн-очереди. Важно

обрабатывать миграцию, когда структура данных изменяется, и защищать чувствительные данные (используя Keychain на iOS, Keystore на Android). Персистентность состояния гарантирует, что пользователи не теряют данные и обеспечивает лучший офлайн-опыт.

26. Объясните лучшие практики безопасности React Native.

Практики безопасности включают: хранение чувствительных данных в безопасном хранилище (Keychain/Keystore), использование certificate pinning для вызовов API, обфускацию кода JavaScript, валидацию и санитизацию всех входных данных, использование HTTPS для всех сетевых запросов, реализацию правильной аутентификации и авторизации, избегание хранения чувствительных данных в AsyncStorage и поддержание зависимостей в актуальном состоянии. Для высокочувствительных приложений вам могут понадобиться дополнительные меры, такие как обнаружение root/jailbreak, защита от модификации и runtime application self-protection (RASP). Безопасность особенно важна для финансовых, медицинских приложений или приложений, обрабатывающих персональные данные.

27. Как вы обрабатываете мониторинг производительности приложения?

Мониторинг производительности включает: отслеживание времени запуска приложения, измерение времени рендеринга экранов, мониторинг производительности потока JavaScript, отслеживание производительности нативных модулей, использование инструментов, таких как Flipper или React Native Debugger, реализацию пользовательских маркеров производительности и использование сервисов, таких как Firebase Performance Monitoring или New Relic. Вы должны выявлять узкие места, оптимизировать медленные экраны, уменьшать размер бандла и мониторить использование памяти. Мониторинг производительности помогает выявлять проблемы до того, как пользователи их испытают, и направляет усилия по оптимизации. Регулярное профилирование необходимо для поддержания хорошей производительности приложения.

28. Какие соображения по управлению памятью в React Native?

Управление памятью включает: правильную очистку подписок и слушателей в cleanup useEffect, избегание утечек памяти из замыканий, соответствующее управление кэшированием изображений, освобождение ресурсов нативных модулей, избегание циклических ссылок и

мониторинг использования памяти. Приложения React Native могут иметь проблемы с памятью из-за: больших изображений, которые не освобождаются, слушателей, которые не удаляются, больших массивов/объектов в состоянии и утечек нативных модулей. Инструменты, такие как профилировщик памяти Flipper, помогают выявлять утечки. Правильная очистка в функциях возврата `useEffect` важна для предотвращения утечек памяти.

29. Как вы обрабатываете обновления приложения и версионирование?

Обновления приложения требуют: семантического версионирования (`major.minor.patch`), ведения changelog, реализации подсказок об обновлении для критических обновлений, использования `code pushing` для обновлений JavaScript, когда возможно, и планирования релизов в магазине приложений. Вы должны: тщательно тестировать обновления, поддерживать обратную совместимость, когда возможно, сообщать об изменениях пользователям и иметь планы отката. Для нативных обновлений вам нужно пройти проверку в магазине приложений, что занимает время. `Code pushing` позволяет более быстрые обновления JavaScript. Управление версиями помогает отслеживать, что развернуто, и позволяет использовать `feature flags` или постепенные развертывания.

30. Объясните workflow разработки React Native и инструменты.

Workflow разработки обычно включает: настройку среды разработки (Node.js, React Native CLI или Expo), использование Metro bundler для бандлинга JavaScript, запуск на симуляторах/эмуляторах или физических устройствах, использование hot reloading для быстрой итерации, отладку с React Native Debugger или Flipper, использование ESLint и Prettier для качества кода, реализацию Git workflow и настройку CI/CD для автоматизированного тестирования и сборки. Основные инструменты включают: VS Code с расширениями React Native, Flipper для отладки, Fastlane для автоматизации развертывания и аккаунты разработчиков в магазинах приложений. Хороший workflow значительно улучшает скорость разработки и качество кода.